NLP · Day 28 of 30 · about 60 minutes

Bag-of-words and TF-IDF

🎯 Today's goal

By the end of this hour you will:

- build a **vocabulary** and turn a text into a row of numbers (a **vector**)
- do the same in one line with scikit-learn's `CountVectorizer`
- use built-in stop words and word pairs (**n-grams**)
- understand **TF-IDF**: which words really matter in a text
- measure how similar two texts are and build a tiny search engine

🤔 Why this matters

Yesterday you cleaned text into tokens. But every model from the machine learning days, from linear regression to neural networks, only understands **numbers**. So we need a way to turn a text into numbers.

The simplest idea: throw all the words of a text into a bag, shake it, and count what's inside. The order is lost ("dog bites man" and "man bites dog" look the same), but the counts already tell a lot about what a text is about. This **bag-of-words** idea, and its smarter cousin TF-IDF, powered search engines and spam filters for years, and they are still a strong first try for any text problem.

🍝 Our data: eight short recipes

Our corpus today is eight **made-up** one-line recipe descriptions. They are already tidy (no punctuation), so we can focus on the new ideas:

```
# 8 made-up recipe descriptions (invented for this lesson)
docs = [
    "Tomato pasta with garlic and fresh basil",
    "Creamy mushroom pasta with garlic and cheese",
    "Chocolate cake with dark chocolate and cream",
    "Spicy chicken curry with rice and garlic",
    "Fresh tomato salad with basil and olive oil",
    "Lemon cake with cream and fresh berries",
    "Chicken noodle soup with garlic and ginger",
    "Vegetable rice with spicy tomato sauce",
]

for number, doc in enumerate(docs):
    print(number, doc)
```

```
0 Tomato pasta with garlic and fresh basil
1 Creamy mushroom pasta with garlic and cheese
2 Chocolate cake with dark chocolate and cream
3 Spicy chicken curry with rice and garlic
4 Fresh tomato salad with basil and olive oil
5 Lemon cake with cream and fresh berries
6 Chicken noodle soup with garlic and ginger
7 Vegetable rice with spicy tomato sauce
```

🎒 Bag-of-words by hand

Step one: the **vocabulary**, the list of every different word in the whole corpus. `set(a_list)` turns a list into a set, which throws away repeats (you met sets yesterday), and `sorted()` puts the words in A–Z order:

```
all_words = []
for doc in docs:
    all_words += doc.lower().split()

vocab = sorted(set(all_words))
print(len(all_words), "words in total,", len(vocab), "different words")
print(vocab)
```

```
56 words in total, 28 different words
['and', 'basil', 'berries', 'cake', 'cheese', 'chicken', 'chocolate', 'cream',
'creamy', 'curry', 'dark', 'fresh', 'garlic', 'ginger', 'lemon', 'mushroom',
'noodle', 'oil', 'olive', 'pasta', 'rice', 'salad', 'sauce', 'soup', 'spicy',
'tomato', 'vegetable', 'with']
```

Step two: describe each document by **counting** every vocabulary word in it. The result is a **vector**: simply a list of numbers, here one number per vocabulary word. Every document gets a vector of

the same length, so a model can compare them position by position:

```
def count_vector(doc):
    words = doc.lower().split()
    return [words.count(word) for word in vocab]

print(docs[2])
print(count_vector(docs[2]))

# Which positions are not zero?
for word, count in zip(vocab, count_vector(docs[2])):
    if count > 0:
        print(f"{word:>10}: {count}")
```

```
Chocolate cake with dark chocolate and cream
[1, 0, 0, 1, 0, 0, 2, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 1]
       and: 1
      cake: 1
 chocolate: 2
     cream: 1
      dark: 1
      with: 1
```

`zip(a, b)` walks through two lists side by side, giving one item from each at a time.

⚙️ The same with scikit-learn: `CountVectorizer`

scikit-learn does all of this in two lines. A **vectorizer** turns texts into vectors. It follows the pattern you know from the models: `fit` learns the vocabulary, `transform` counts the words, and `fit_transform` does both at once. It also lowercases and tokenises for you.

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(docs)

print(X.shape)        # (documents, vocabulary words)
print(vectorizer.get_feature_names_out())
```

```
(8, 28)
['and' 'basil' 'berries' 'cake' 'cheese' 'chicken' 'chocolate' 'cream'
 'creamy' 'curry' 'dark' 'fresh' 'garlic' 'ginger' 'lemon' 'mushroom'
 'noodle' 'oil' 'olive' 'pasta' 'rice' 'salad' 'sauce' 'soup' 'spicy'
 'tomato' 'vegetable' 'with']
```

Eight documents, 28 words: the same vocabulary we built by hand. `X` is a **sparse matrix**. Most of its numbers are zero (each recipe uses only a few of the words), so to save memory it only stores the numbers that aren't zero. `.toarray()` turns it into a normal NumPy array. Wrapped in a DataFrame it becomes the **document-term matrix**: one row per document, one column per word. Here are six of the 28 columns:

```
import pandas as pd

dtm = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
dtm[["garlic", "pasta", "tomato", "cake", "chocolate", "rice"]]
```

	garlic	pasta	tomato	cake	chocolate	rice
0	1	1	1	0	0	0
1	1	1	0	0	0	0
2	0	0	0	1	2	0
3	1	0	0	0	0	1
4	0	0	1	0	0	0
5	0	0	0	1	0	0
6	1	0	0	0	0	0
7	0	0	1	0	0	1

Row 2 has `chocolate = 2`: "chocolate cake with dark chocolate". Once the vectorizer has learned its vocabulary, use `transform` on new texts. Watch out for this classic mistake:

💡 This example has a mistake on purpose. Run it and read the error.

```
vectorizer.transform("garlic bread with garlic butter")
```

⚠️ **ValueError**

The kind of value is right but the value itself isn't, like int("hello").

```
Traceback (most recent call last):
  line 1, in <module>
    vectorizer.transform("garlic bread with garlic butter")
    ~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: Iterable over raw text documents expected, string object received.
```

"Iterable over raw text documents expected, string object received": a vectorizer always wants a **list** of documents, even if there is only one. Put square brackets around the text:

```
new = vectorizer.transform(["garlic bread with garlic butter"])
new_counts = pd.Series(new.toarray()[0], index=vectorizer.get_feature_names_out())

new_counts[new_counts > 0]      # only the words it knows
```

```
garlic    2
with      1
dtype: int64
```

`bread` and `butter` are missing: the vectorizer only counts words from the vocabulary it learned during `fit`. New words are simply ignored.

🛑 Built-in stop words

Yesterday you wrote a stop-word list by hand. scikit-learn has an English list built in (it needs no download), switched on with `stop_words="english"`:

```
plain = CountVectorizer().fit(docs)
no_stop = CountVectorizer(stop_words="english").fit(docs)

print(len(plain.get_feature_names_out()), "words without the stop-word list")
print(len(no_stop.get_feature_names_out()), "words with it")

removed = set(plain.get_feature_names_out()) - set(no_stop.get_feature_names_out())
print("removed:", sorted(removed))
```

```
28 words without the stop-word list
26 words with it
removed: ['and', 'with']
```

Taking one set away from another (`set_a - set_b`) keeps the items that are only in the first.

🔗 Word pairs: n-grams

A bag of single words loses the order, so *"not good"* becomes just *not* and *good*. An **n-gram** is a group of *n* words in a row: a **unigram** is one word, a **bigram** two. With `ngram_range=(1, 2)` the vectorizer counts both:

```
reviews = ["the soup was not good", "the soup was good"]

pairs = CountVectorizer(ngram_range=(1, 2))
pairs.fit(reviews)
print(pairs.get_feature_names_out())
```

```
['good' 'not' 'not good' 'soup' 'soup was' 'the' 'the soup' 'was'
 'was good' 'was not']
```

Now `not good` is its own column, so a model can learn that it means something different from `good`. The price: many more columns.

⚖️ TF-IDF: which words really matter?

Plain counts have a weakness. `garlic` appears in four of our eight recipes, so it doesn't tell recipes apart very well. `chocolate` appears in only one recipe, so it says a lot about that recipe. **TF-IDF** gives each word a score that rewards both:

Part	Stands for	Question it answers
TF	term frequency	How often is the word in *this* document? More often → bigger.
IDF	inverse document frequency	In how many documents does the word appear? In fewer documents → bigger.
TF-IDF	TF × IDF	Is the word frequent here *and* rare elsewhere? Then it's important here.

`TfidfVectorizer` works exactly like `CountVectorizer`, but gives these scores instead of counts. (It also scales each row so long and short documents can be compared fairly.) Let's look at the IDF part first:

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(stop_words="english")
T = tfidf.fit_transform(docs)
words = tfidf.get_feature_names_out()

idf = pd.Series(tfidf.idf_, index=words).round(2)
idf[["garlic", "tomato", "pasta", "chocolate"]]
```

```
garlic       1.59
tomato       1.81
pasta        2.10
chocolate    2.50
dtype: float64
```

`garlic` (in 4 recipes) gets the smallest IDF, `chocolate` (in 1) the biggest. Now the full TF-IDF scores of recipe 0, "Tomato pasta with garlic and fresh basil", sorted from most to least important:

```
scores = pd.Series(T.toarray()[0], index=words)
scores[scores > 0].sort_values(ascending=False).round(2)
```

```
basil     0.50
pasta     0.50
fresh     0.43
tomato    0.43
garlic    0.38
dtype: float64
```

Each word appears once, but `pasta` and `basil` score highest because few other recipes use them; `garlic` scores lowest. The top TF-IDF words make a good short summary of a document: they are its **keywords**.

🧭 How similar are two texts?

Now that every recipe is a vector, we can compare them. **Cosine similarity** compares two vectors and gives a score from **0** (no words in common) to **1** (the same words in the same proportions). The name comes from geometry: it measures the angle between the two vectors, but you don't need the maths to use it. `cosine_similarity(T)` compares every document with every other one:

```
from sklearn.metrics.pairwise import cosine_similarity

similarity = pd.DataFrame(cosine_similarity(T)).round(2)
similarity
```

	0	1	2	3	4	5	6	7
0	1.00	0.32	0.00	0.13	0.48	0.16	0.12	0.16
1	0.32	1.00	0.00	0.11	0.00	0.00	0.10	0.00
2	0.00	0.00	1.00	0.00	0.00	0.28	0.00	0.00
3	0.13	0.11	0.00	1.00	0.00	0.00	0.29	0.38
4	0.48	0.00	0.00	0.00	1.00	0.12	0.00	0.12
5	0.16	0.00	0.28	0.00	0.12	1.00	0.00	0.00
6	0.12	0.10	0.00	0.29	0.00	0.00	1.00	0.00
7	0.16	0.00	0.00	0.38	0.12	0.00	0.00	1.00

The diagonal is 1 (every recipe is identical to itself). Recipe 0 (tomato pasta) is most similar to recipe 4 (tomato salad, 0.48): they share *tomato*, *fresh* and *basil*. Recipe 2 (chocolate cake) only resembles recipe 5 (lemon cake).

That's all a basic **search engine** needs: turn the question (the **query**) into a vector with the *same* vectorizer, compare it with every document, and pick the highest score. NumPy's `.argmax()` (Day 9) gives the position of the biggest number:

```
query = "something spicy with rice"

query_vector = tfidf.transform([query])
match_scores = cosine_similarity(query_vector, T)[0]   # [0]: the first (only) query
print(match_scores.round(2))

best = match_scores.argmax()
print(f"Best match: recipe {best}, {docs[best]!r} (score {match_scores[best]:.2f})")
```

```
[0.   0.   0.   0.63 0.   0.   0.   0.6 ]
Best match: recipe 3, 'Spicy chicken curry with rice and garlic' (score 0.63)
```

`{docs[best]!r}` shows the text with quotes around it. Try your own queries, like `"a sweet cake"` or `"pasta with cheese"`.

⚠️ Common mistakes

- Passing one string instead of a list: `transform("text")` → `ValueError`. Use `transform(["text"])`.
- Calling `fit` or `fit_transform` again on the query. Fit **once** on the documents, then only `transform` new texts, or the columns won't match.
- Expecting new words to be counted: words not in the vocabulary are ignored.
- Forgetting that `X` is a sparse matrix: use `.toarray()` to see the numbers.
- Reading a cosine similarity of 0.3 as "30% the same text". It's a score for comparing, not a percentage.

✅ Recap

- The **vocabulary** is every different word in the corpus; a **vector** is a list of numbers, one per word.
- **Bag-of-words**: count each word, ignore the order. `CountVectorizer().fit_transform(docs)`.
- `get_feature_names_out()` lists the words; `.toarray()` shows the numbers.
- `stop_words="english"` removes common words; `ngram_range=(1, 2)` adds word pairs.
- **TF-IDF** = frequent here × rare elsewhere. `TfidfVectorizer` gives these scores.
- `cosine_similarity` scores how alike two vectors are (0 to 1); the highest score is the best match.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Build a vocabulary

Store the sorted list of the different words in these three documents in a variable called `vocab`. Do it by hand (split, `set`, `sorted`) or with `CountVectorizer`.

```
docs = ["red apple pie", "green apple juice", "red berry juice"]

# Store the sorted list of different words in a variable called vocab
```

Hints:

1. Collect all the words: `all_words = []`, then for each doc `all_words += doc.split()`.
2. `set(all_words)` removes the repeats; `sorted(...)` gives an A–Z list.
3. `vocab = sorted(set(all_words))`

2. A count matrix

Use `CountVectorizer` on the three documents, then store:

- `n_docs` and `n_words`: the two numbers in `X.shape`,
- `cheese_count`: how many times `cheese` appears in the **first** document, read from `X`.

```
from sklearn.feature_extraction.text import CountVectorizer

docs = ["cheese pizza with extra cheese", "tomato soup", "cheese and tomato
toast"]

# 1. Make X with CountVectorizer
# 2. Store n_docs and n_words
# 3. Store cheese_count
```

Hints:

1. `vectorizer = CountVectorizer()` and `X = vectorizer.fit_transform(docs)`
2. `n_docs, n_words = X.shape`
3. Find the column of "cheese": `column = list(vectorizer.get_feature_names_out()).index("cheese")`, then `cheese_count = X.toarray()[0, column]`.

3. The most important word

Use `TfidfVectorizer()` (no options) on the four documents. Store the word with the **highest TF-IDF score in the first document** in a variable called `top_word`.

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

docs = ["garlic bread with garlic butter", "garlic soup", "tomato soup with
bread", "apple pie"]

# Store the top word of the first document in top_word
```

Hints:

1. `tfidf = TfidfVectorizer()` and `T = tfidf.fit_transform(docs)`
2. Put the first row in a Series: `scores = pd.Series(T.toarray()[0], index=tfidf.get_feature_names_out())`
3. `scores.idxmax()` gives the index label (the word) of the biggest value. Or use `scores.sort_values(ascending=False).index[0]`.

4. The most similar document

Which document is most similar to the query? Fit a `TfidfVectorizer()` on the documents, transform the query, compare with `cosine_similarity`, and store the **position** (index) of the best document in `best_index`.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = ["hot chicken soup", "cold tomato salad", "hot tomato soup with bread",
"fresh bread rolls"]
query = "tomato soup"

# Store the index of the most similar document in best_index
```

Hints:

1. `tfidf = TfidfVectorizer()`, `T = tfidf.fit_transform(docs)`, `q = tfidf.transform([query])`
2. `scores = cosine_similarity(q, T)[0]` gives one score per document.
3. `best_index = scores.argmax()`

5. Challenge: a movie search engine

Write a function `search(query)` that returns the **text** of the made-up movie description that best matches the query. Fit a `TfidfVectorizer()` (no options) on `movies` once, outside the function; inside, transform the query, compare, and return the best description. The checker will try several queries.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Made-up movie descriptions
movies = [
    "a robot learns to love on a lonely space station",
    "two friends open a bakery in a small seaside town",
    "a detective hunts a thief across a snowy city",
    "astronauts fight to survive after their space ship breaks",
    "a young chef enters a baking contest in paris",
]

def search(query):
    # return the movie description that best matches the query
    return ""

print(search("space robot"))
```

Hints:

1. Before the function: `tfidf = TfidfVectorizer()` and `T = tfidf.fit_transform(movies)`.
2. Inside: `scores = cosine_similarity(tfidf.transform([query]), T)[0]`
3. `return movies[scores.argmax()]`

➡️ Tomorrow

A sentiment classifier: TF-IDF vectors are perfect inputs for the classification models from the machine learning days. Tomorrow you'll train a model on 80 made-up labelled reviews so it can tell on its own whether a new review is positive or negative, and find out which words it relies on.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Build a vocabulary

```
docs = ["red apple pie", "green apple juice", "red berry juice"]

all_words = []
for doc in docs:
    all_words += doc.split()

vocab = sorted(set(all_words))
print(vocab)
```

2. A count matrix

```
from sklearn.feature_extraction.text import CountVectorizer

docs = ["cheese pizza with extra cheese", "tomato soup", "cheese and tomato
toast"]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(docs)

n_docs, n_words = X.shape

column = list(vectorizer.get_feature_names_out()).index("cheese")
cheese_count = X.toarray()[0, column]
print(n_docs, n_words, cheese_count)
```

3. The most important word

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

docs = ["garlic bread with garlic butter", "garlic soup", "tomato soup with
bread", "apple pie"]

tfidf = TfidfVectorizer()
T = tfidf.fit_transform(docs)
scores = pd.Series(T.toarray()[0], index=tfidf.get_feature_names_out())
print(scores.sort_values(ascending=False))

top_word = scores.idxmax()
print(top_word)
```

4. The most similar document

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = ["hot chicken soup", "cold tomato salad", "hot tomato soup with bread",
"fresh bread rolls"]
query = "tomato soup"

tfidf = TfidfVectorizer()
T = tfidf.fit_transform(docs)
q = tfidf.transform([query])

scores = cosine_similarity(q, T)[0]
print(scores.round(2))
best_index = scores.argmax()
print(best_index, docs[best_index])
```

5. Challenge: a movie search engine

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Made-up movie descriptions
movies = [
    "a robot learns to love on a lonely space station",
    "two friends open a bakery in a small seaside town",
    "a detective hunts a thief across a snowy city",
    "astronauts fight to survive after their space ship breaks",
    "a young chef enters a baking contest in paris",
]

tfidf = TfidfVectorizer()
T = tfidf.fit_transform(movies)

def search(query):
    scores = cosine_similarity(tfidf.transform([query]), T)[0]
    return movies[scores.argmax()]

print(search("space robot"))
```